Microcontrollers & Embedded Systems
(MTS-311)
DE-45 Mechatronics
Syndicate – C

Ánimo Platte: A Mood-Responsive Environment Controller

Project Report

1. **461653, Ebad Naeem**: Testing & Documentation Lead
2. **463831, Farhan Shahid:** Control & Algorithms Lead
3. **460960, Khadija Siddiqui**: Simulation, Logging, & Integration Lead
4. **457039, Khizer Kashif**: Electronics Lead
5. **467289, Mahd Hassan**: Firmware & Coding Lead

Date of Submission: **8th December 2025**	Submitted to: **Dr Zohaib Riaz**

Table of Contents

1.0. Project Description

Ánimo Platte is a Mood-Responsive Environment Controller that senses occupant comfort (temperature, humidity, light, sound, movement) and indoor air quality (CO_2, VOCs). The microcontroller dynamically adjusts lighting (PWM), fan speed (PWM), ventilation servo, purifier servo, and background ambience using a fuzzy rule set to maximize comfort and air quality while minimizing energy use.

2.0. Objectives

The main objectives of the Ánimo Platte system are:

- To develop a multi-parameter smart environment controller capable of real-time adaptive decision-making.
- To integrate fuzzy logic controllers (FLCs) for lighting, climate, noise, and air quality.
- To incorporate occupancy awareness using a PIR-based interrupt-driven override.
- To implement closed-loop actuation of RGB lighting, DC fan, ventilation servo, purifier servo, and ambient buzzer.
- To support manual override through Blynk (ESP8266 serial commands).
- To design a stable, safe, and fully embedded hardware integration on the Arduino UNO WiFi R3 microcontroller.
- To ensure comprehensive testing of each subsystem using individual sensor test codes.

3.0. Theoretical Framework

3.1. BH1750 Light Sensor

The BH1750 is a digital light intensity sensor that uses the I2C communication protocol. It outputs luminosity measurements in lux, the SI unit for measuring light. It can measure a minimum of 1 lux and a maximum of 65535 lux. The BH1750 has a number of features that make it a versatile and reliable light sensor [1]. It supports multiple measurement modes and is ideal for fuzzy-lighting systems.

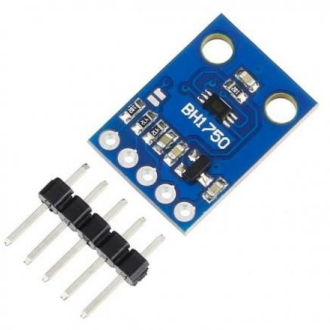


Figure 1. BH1750 Module
Adapted from [1]

3.2. BME280 Temperature & Humidity Sensor

The BME280 is a combined digital humidity, pressure and temperature sensor based on proven sensing principles. Its small dimensions and its low power consumption allow the implementation in battery-driven devices such as handsets, GPS modules or watches. Anything you need to know about atmospheric conditions, you can find out from this tiny breakout. The BME280 Breakout has been designed to be used in indoor/outdoor navigation, weather forecasting, home automation, and even personal health and wellness monitoring. This sensor is great for all sorts of weather sensing and can even be used in both I2C and SPI [2].

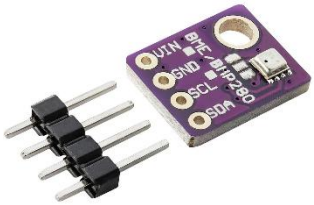


Figure 2. BME280 Module
Adapted from [2]

3.3. MAX9814 Condenser Microphone Amplifier

The MAX9814 module is a microphone amplifier with automatic gain control (AGC). It is a popular choice for applications where the loudness of the audio signal is not predictable, such as voice recognition, noise cancellation, and audio recording.

The AGC circuit automatically adjusts the gain of the amplifier to keep the output signal at a constant level, regardless of the input signal level. This helps to prevent clipping and distortion, even when the input signal level is very high or very low [3].

Figure 3. MAX9814 Module
Adapted from [3]

3.4. MQ-135 Air Quality Sensor

MQ135 Gas Sensor is an Air Quality Sensor for detecting a wide range of gases, including CO2, NOx, alcohol, benzene, smoke and NH3. MQ135 Gas sensors are used in Air Quality Control equipment and are suitable for detecting or measuring NH3, NOx, Alcohol, Benzene, Smoke, CO2.

The MQ135 Gas Sensor is a low-cost, easy-to-use sensor that can use to detect a variety of harmful gases, including ammonia, nitrogen dioxide, sulfur dioxide, and benzene. It is a metal oxide semiconductor (MOS) sensor, which means that it uses a thin film of tin dioxide (SnO2) to detect the presence of gases [4].

Figure 4. MQ-135 Module
Adapted from [4]

3.5. PIR Motion Sensor (HW-416-B)

The PIR sensor stands for Passive Infrared sensor. A PIR sensor detects changes in incoming infra-red (IR) radiation and outputs a signal. They are typically tuned to the frequency of IR that corresponds to normal human body temperature and so they make good motion detectors. They are not completely dependent on live people or creatures, though, since any movement will cause changes in the ambient IR field through occlusion, reflection, etc [5].

Figure 5. HW-416-B PIR Sensor
Adapted from [5]

3.6. RGB LED Module (HW-478)

The KY-009 RGB LED SMD module is a small, inexpensive electronic component that can be used to create and display colors. KY-009 consists of an RGB LED (red, green, blue) and a control switch that controls the LED. The LED

can produce a variety of colors by varying the brightness of the three primary colors.

RGB LED SMD module can use with a variety of microcontrollers, including Arduino, Raspberry Pi, and ESP8266. There are many libraries available for these microcontrollers that make it easy to control the KY-009 RGB LED SMD module. The KY-009 RGB LED SMD module is a versatile and affordable option for creating and displaying colors. It is easy to use and can be controlled with a variety of microcontrollers [6].

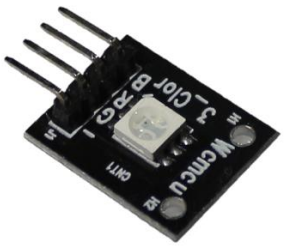


Figure 6. KY-009 RGB SMD LED Module
Adapted from [6]

3.7. Actuators

- DC Cooling Fan (PWM speed control)
- SG90 Purifier Servo (crisp ON/OFF based on pollutant spike)
- SG90 Ventilation Servo (position mapped to fuzzy PWM output)
- Buzzer (sound ambience control using PWM)

3.8. Arduino UNO WiFi R3 (ATmega328P + ESP8266 8Mb)

The Arduino Uno Wi-Fi is the same as an Arduino Uno Rev3 but with an integrated Wi-Fi module. The board is based on the ATmega328P with an integrated ESP8266 Wi-Fi Module. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button.

Figure 7. Arduino UNO WiFi R3 ATmega328P + ESP8266 8Mb USB-TTL CH340G Development Board

3.9. Fuzzy Logic Control Theory (FLC)

Ánimo Platte implements four FLCs + occupancy logic:

1. Light FLC (uses BH1750 lux)
2. Noise FLC (uses MAX9814 peak amplitude)
3. Climate FLC (temperature & humidity → fan PWM)
4. Air Quality FLC (MQ-135 → ventilation PWM)
5. Occupancy Override (PIR interrupt → full system suppression)

Each FLC performs:
Fuzzification → Rule Inference → Aggregation → Defuzzification → Actuation.

Membership Functions: All membership functions (triangular, trapezoidal), rule tables, and defuzzification calculations were written on paper. The pictures for these are provided in Appendix A.

3.10. Libraries Used in Firmware

The following software libraries were used in the Ánimo Platte firmware. Each library provides essential abstractions and hardware interface layers required for sensing, actuation, and control logic.

3.10.1. Wire.h

Enables I²C communication between the microcontroller and digital sensors such as the BH1750 and BME280. Used as the underlying communication layer for all I²C operations.

3.10.2. BH1750_WE.h

A dedicated library for the BH1750 digital light sensor. It simplifies lux measurement, provides stable calibration, and supports continuous and one-time measurement modes.

3.10.3. Adafruit_Sensor.h

A unified sensor interface used by multiple Adafruit libraries. Provides standardized data structures and ensures cross-compatibility and consistent measurement handling.

3.10.4. Adafruit_BME280.h

Used to communicate with the BME280 environmental sensor. Provides accurate temperature, humidity, and pressure readings and manages sensor initialization, calibration, and data retrieval.

3.10.5. Servo.h

Controls the SG90 servos used in the purifier and ventilation system. Maps PWM timing values to servo angles (0°–180°) and manages pulse generation.

3.10.6. (Built-In) Arduino Core Libraries

Includes functions for GPIO handling, PWM generation, ADC sampling, timers, and serial communication. These are essential for reading analog sensors, driving actuators, and processing ESP8266 serial commands.

4.0. Hardware Used and Bill of Materials (BoM)

The Ánimo Platte system consists of both prefabricated electronic modules and custom-designed hardware. The following table lists all components used in the project along with their quantities and estimated costs.

BoM Level	Description	Qty	Units	Unit Cost	Total Cost
1	Arduino UNO WiFi R3 (ATmega328P + ESP8266)	1	1	1,650 PKR	1,650 PKR
2	BH1750 Module	1	2	350 PKR	700 PKR
3	BME280 Module	1	1	850 PKR	850 PKR
4	MAX9814 Module	1	1	600 PKR	600 PKR
5	MQ-135 Module	1	2	320 PKR	640 PKR
6	PIR Sensor HW-416-B	1	1	200 PKR	200 PKR
7	RGB Module HW-478	1	2	60 PKR	120 PKR
8	SG90 Servos (Purifier + Vent)	2	2	300 PKR	600 PKR
9	DC Cooling Fan	1	2	250 PKR	500 PKR
10	IRF-530 MOSFET	1	2	90 PKR	180 PKR
11	1N4007 Diode	1	10	3 PKR	30 PKR
12	10k Ω Resistors	1	10	2 PKR	20 PKR
13	2 Pin Screw Terminals	1	2	20 PKR	40 PKR
11	Copper Sheet	1	—	450 PKR	450 PKR
12	$FeCl_3$	—	—	250 PKR	250 PKR
13	Header pins, wires	—	—	150 PKR	150 PKR
Total Number of Parts	**13**				
Total Cost					**6,980 PKR**

5.0. System Design and Methodology

5.1. High-Level Architecture

All subsystems follow a 1-second FLC cycle.

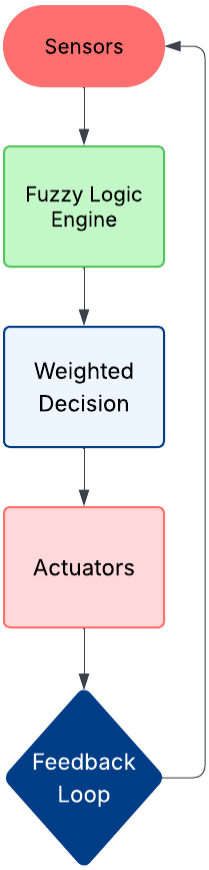


Figure 8. System Level Flowchart

5.2. Sensor Interfacing Overview

- BH1750 via I²C (A4/A5)
- BME280 via I²C
- MAX9814 via analog input A1
- MQ-135 via analog input A0
- PIR via digital interrupt pin D2
- RGB LEDs via PWM pins D9–D11
- Fan via MOSFET driver using D5
- Servos attached to D6 and D13

5.3. Fuzzy Logic Membership Functions

Each sensor had its own fuzzy logic membership function. The detailed pictures of the formulation of these membership functions is given in Appendix A.

5.4. Control Architecture

5.4.1. Automatic (FLC Mode)

All actuators are driven by fuzzy rules.

5.4.2. Manual (Blynk Serial Mode)

ESP8266 sends commands like:

```
V1:120
V7:200
V9:1
```

UNO interprets → overrides subsystem.

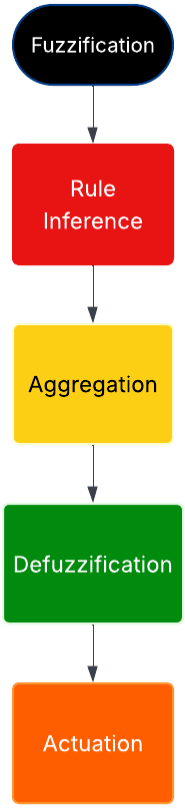


Figure 9. Control Flowchart

5.5. Occupancy Override

If PIR = LOW → ∀ actuators OFF.

6.0. Practical Implementation

6.1. Hardware Assembly

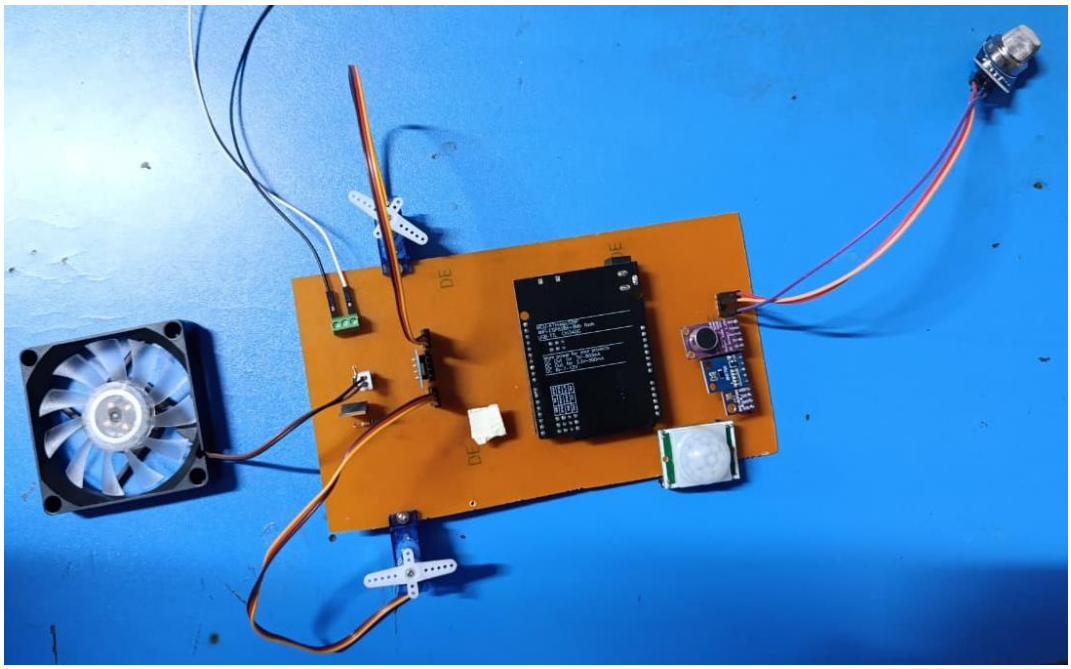


Figure 10. Hardware Assembly

6.2. Sensor Test Snippets

6.2.1. BH1750 Test

```
#include <BH1750.h>

BH1750 light;

void setup(){Wire.begin();light.begin();Serial.begin(9600);}

void loop(){Serial.println(light.readLightLevel());delay(500);}
```

6.2.2. BME280 Test

```
#include <Adafruit_BME280.h>

Adafruit_BME280 bme;

void setup(){bme.begin(0x76);}

void loop(){Serial.println(bme.readTemperature());}
```

6.2.3. MQ-135 Test

```
void loop(){int v=analogRead(A0);Serial.println(v);delay(300);}
```

6.2.4. MAX9814 Test

```
int mic=A1;void loop(){Serial.println(analogRead(mic));}
```

6.2.5. PIR Test Snippet

```
void loop(){Serial.println(digitalRead(2));}
```

7.0. Results

The results for the project are shown in the demo videos, the links to which are provided in the Appendix E.

8.0. Limitations & Future Improvements

8.1. Limitations

- The MQ-135 sensor provides a generalized air-quality reading (CO_2, VOCs, NH_3, smoke) but lacks calibrated CO_2 PPM accuracy.
- The UNO–ESP8266 serial-bridge architecture introduces latency during heavy serial communication.
- The system currently relies on a simple buzzer for ambience rather than a fully tunable sound-based mood generator.
- Servos may jitter under high PWM demand due to UNO's timer limitations.
- Environmental states are controlled individually but not yet fused into a higher-level "mood profile" (Relax, Focus, Sleep, Active).

8.2. Future Improvements

- Integrate a dedicated CO_2 sensor (e.g., MH-Z19B) to obtain precise indoor air-quality metrics.
- Replace UNO + ESP8266 with a single ESP32 for reduced latency, more timers, and smoother servo control.
- Add an OLED dashboard for real-time visualization without needing Blynk.

- Implement environment-based presets (e.g., Relax Mode, Focus Mode, Sleep Mode) combining lighting, sound, airflow, and temperature preferences.
- Expand fuzzy rules to support user mood detection and adaptive ambience generation.
- Add machine-tunable gain values for all controllers to streamline calibration.

9.0. Conclusion

Ánimo Platte successfully integrates fuzzy logic, environmental sensing, and context-aware actuation to create a smart, energy-efficient comfort management system. With full support for manual override, occupancy suppression, and multi-sensor fusion, it demonstrates a strong embedded system capable of adaptive decision-making in real indoor environments.

10.0. References

[1] Majju, "GY-30 BH1750 Digital Ambient Light Intensity Sensor Module," [Online]. Available: https://www.majju.pk/product/bh1750-digital-light-intensity-sensor-module-for-arduino/#:~:text=Features%20:%20%E2%80%93,I2C%20communication. [Accessed 6 December 2025].

[2] Bosch, "BME280 Datasheet," [Online]. Available: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf. [Accessed 6 December 2025].

[3] Majju, "MAX9814 Module Electret Microphone Amplifier Board," [Online]. Available: https://www.majju.pk/product/max9814-module-electret-microphone-amplifier-board/?srsltid=AfmBOopVf51kZycJUnV6nuUFfx1-Ci6KhClTB4sYhZbcQZQO32xOlzDj. [Accessed 6 December 2025].

[4] Majju, "MQ135 Gas Sensor CO2 Air Quality Sensor," [Online]. Available: https://www.majju.pk/product/mq135-gas-sensor-mq-135-air-quality-sensor-for-arduino/?srsltid=AfmBOorxP2jJrRNncUOw5MeOyufm2n9Hs0NJaAJOGkrrdvfI2jXogLgZ. [Accessed 7 December 2025].

[5] B. Davison, "Passive infra-red (PIR)," [Online]. Available: https://bdavison.napier.ac.uk/iot/Tutorials/Particle/components/pir/. [Accessed 6 December 2025].

[6] Majju, "KY-009 RGB SMD LED Module Programmable 4-Pin," [Online]. Available: https://www.majju.pk/product/ky-009-rgb-smd-led-module-programmable-4-pin/?srsltid=AfmBOooXiKSTLSqK1NhK87TG2vtdWRVaFl8fOs3VkEgmhGB4jvPMkqlf. [Accessed 6 December 2025].

11.0. Appendices

A. Handwritten Membership Functions & Defuzzification

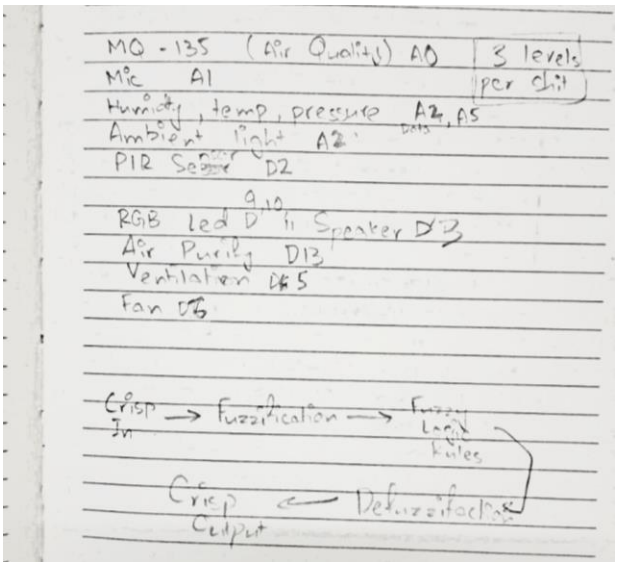


Figure 11. Algorithm Making Process a)

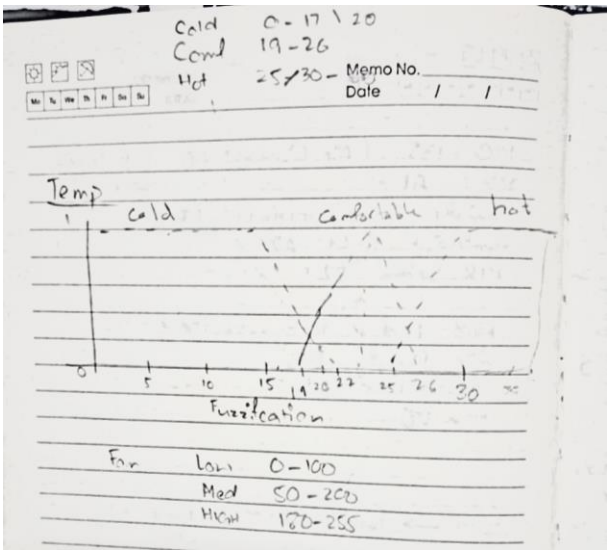
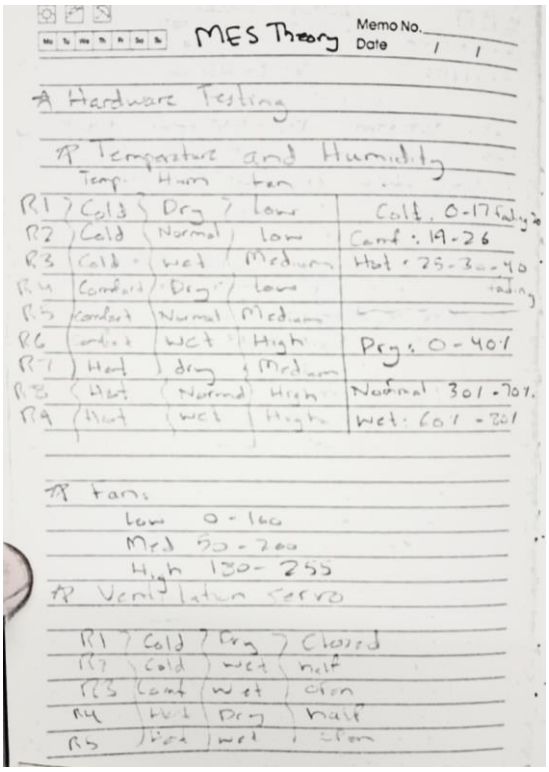


Figure 11. b)



MES Theory Memo No. Date / /

Hardware Testing

Temperature and Humidity

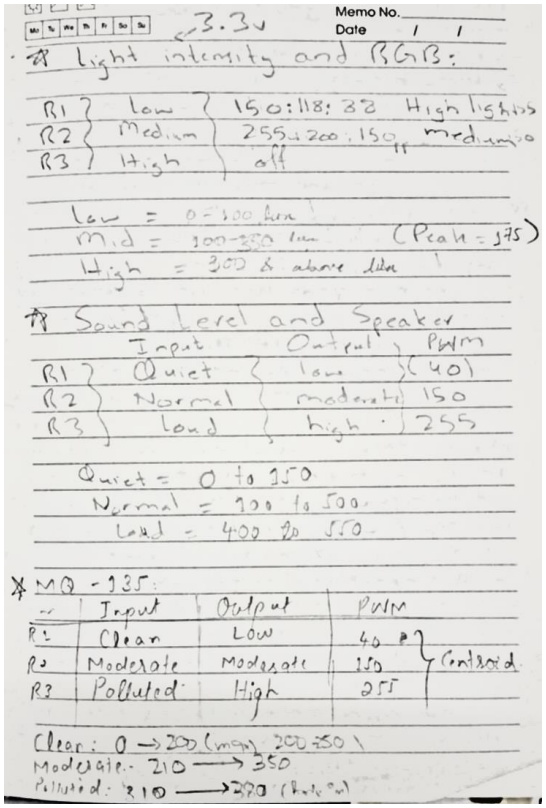
	Temp	Hum	fan	
R1	Cold	Dry	low	Colt. 0-17
R2	Cold	Normal	low	Comf: 19-26
R3	Cold	wet	Medium	Hot: 25-30-40
R4	Comfort	Dry	low	
R5	Comfort	Normal	Medium	
R6	Comfort	Wet	High	Prg: 0-401
R7	Hot	dry	Medium	
R8	Hot	Normal	High	Normal 301-701
R9	Hot	wet	High	wet: 601-801

Fans:
low 0-100
Med 50-200
High 130-255

Ventilation servo

R1	Cold	Dry	Closed
R2	Cold	wet	half
R3	Cool	wet	open
R4	Hot	Dry	half
R5	Hot	wet	open

Figure 11. c)



3.3v Memo No. Date / /

Light intensity and RGB:

R1	Low	150:118:88	High lights
R2	Medium	255:200:150	medium
R3	High	off	

Low = 0-300 lux
Mid = 300-350 lux (Peak = 375)
High = 300 & above lux

Sound Level and Speaker

	Input	Output	Pwm
R1	Quiet	low	40
R2	Normal	moderate	150
R3	loud	high	255

Quiet = 0 to 350
Normal = 300 to 500
Loud = 400 to 550

MQ-135:

	Input	Output	Pwm	
R1	Clean	Low	40	Centroid
R2	Moderate	Moderate	150	
R3	Polluted	High	255	

Clear: 0 → 200 (max 200-350)
Moderate: 210 → 350
Polluted: 310 → 500 (Peak)

Figure 11. d)

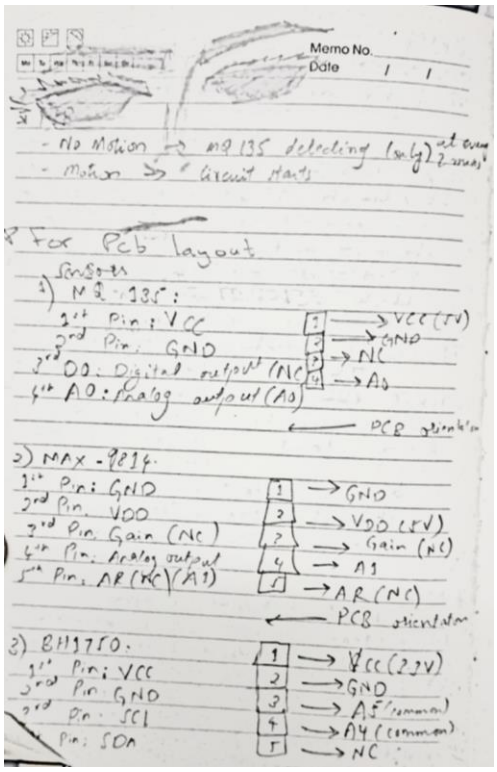


Figure 11. e)

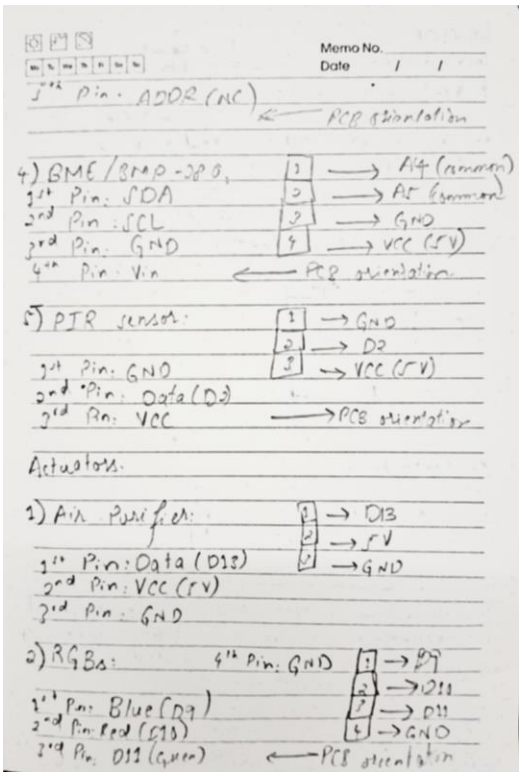


Figure 11. f)

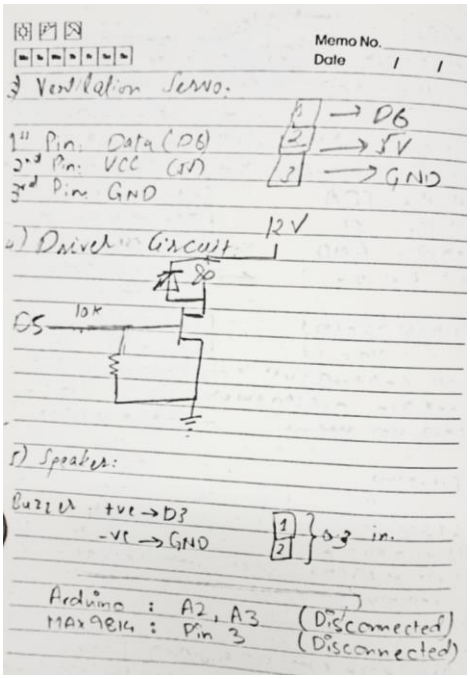


Figure 11. g)

B. Complete Source Code

ATmega 328P Code:

```
// ======================================
// --- FLC LIBRARIES & HARDWARE SETUP ---
// ======================================
#include <Servo.h>
#include <Wire.h>
#include <BH1750_WE.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

// Communication Baud Rate
#define BAUD_RATE 115200

// --- ACTUATOR PIN CONFIGURATION ---
// FLC 5 Occupancy
const int PIN_PIR_SENSOR = 2; // Digital Pin D2 for PIR input (HW-416b) - Connected to Interrupt 0

// FLC 2 Noise
const int PIN_MAX9814_ANALOG = A1;
```

```
const int PIN_SPEAKER_VOLUME_PWM = 3; // V3

// FLC 3/4 Climate/Air Quality
const int PIN_FAN_PWM = 5;          // V7 - Fan (D5) - RESOLVED PIN:
Used D5 for V7 Fan
const int PIN_VENTILATION_SERVO = 6; // V10 - Servo Pin D6
(Ventilation flap) - RESOLVED PIN: Used D6 for V10 Vent
#define BME_ADDRESS 0x76

// FLC 4 Air Quality
const int GAS_SENSOR_PIN = A0;       // MQ-135 Analog Output
const int PURIFIER_SERVO_PIN = 13;    // V9 - Pin D13 for Crisp
Purifier Servo

// FLC 1 Light
#define LED_RED_PIN 10
#define LED_GREEN_PIN 11
#define LED_BLUE_PIN 9 // V1

// --- SENSOR OBJECTS ---
BH1750_WE lightMeter(0x23);
Adafruit_BME280 bme;
Servo PurifierServo;    // V9 Servo object
Servo VentilationServo; // V10 Servo object

// --- FLC 5: GLOBAL VOLATILE STATE & TIMING VARIABLES ---
volatile bool is_occupied = false;
const unsigned long AQ_CHECK_INTERVAL_MS = 120000; // 2 minutes
unsigned long lastAQCheckTime = 0;
int cached_pwm_air_quality = 0;

// =======================================================
// --- CONTROL AUTHORITY FLAGS & MANUAL VALUE STORAGE ---
// =======================================================
enum ControlMode { MODE_FLC, MODE_MANUAL };

// Control Flags: Default to FLC (Automatic) mode
ControlMode lightMode = MODE_FLC;    // V1 (Slider) controls manual
override
ControlMode noiseMode = MODE_FLC;    // V3 (Slider) controls manual
override
ControlMode fanMode = MODE_FLC;      // V7 (Slider) controls manual
override
ControlMode purifierMode = MODE_FLC; // V9 (Switch) controls manual
override
```

```
ControlMode ventMode = MODE_FLC;     // V10 (Slider) controls manual
override

// Manual Value Storage (updated by serial commands)
int manualLightPwm = 0;
int manualNoisePwm = 0;
int manualFanPwm = 0;
int manualPurifierAngle = 0; // 0 or 180 (from V9 switch)
int manualVentAngle = 90;    // 0 to 180 (from V10 slider)

// Serial buffer variables
String inputString = "";

// ============================================
// --- FLC 5: INTERRUPT SERVICE ROUTINE (ISR) ---
// ============================================
void occupancy_change_isr() {
  is_occupied = (digitalRead(PIN_PIR_SENSOR) == HIGH);
  // NOTE: This ISR does not set control modes or actuate pins.
}

// ============================================
// --- FUZZY LOGIC TOOLKIT (Shared Functions) ---
// ============================================

float mu_triangular(float x, float a, float b, float c) {
  if (x <= a || x >= c) return 0.0;
  if (x == b) return 1.0;
  if (x > a && x < b) return (x - a) / (b - a);
  if (x > b && x < c) return (c - x) / (c - b);
  return 0.0;
}

float mu_trapezoidal(float x, float a, float b, float c, float d) {
  if (x >= b && x <= c) {
    return 1.0;
  }
  if (x >= a && x < b) {
    return (x - a) / (b - a);
  }
  if (x > c && x <= d) {
    return (d - x) / (d - c);
  }
  if (x < a || x > d) {
    return 0.0;
```

```
  }
  return 0.0;
}


// ====================================================
// --- FLC 1: LIGHT CONTROL (BH1750) CONFIGURATION ---
// ====================================================
const float LOW_TO_PEAK = 0.0;
const float LOW_TO_ZERO = 120.0;
const float MEDIUM_START = 100.0;
const float MEDIUM_PEAK = 200.0;
const float MEDIUM_END = 350.0;
const float HIGH_START = 300.0;
const float HIGH_TO_MAX = 400.0;

enum LightLinguistic { LIGHT_LOW, LIGHT_MEDIUM, LIGHT_HIGH,
LIGHT_COUNT };
float lightMembership[LIGHT_COUNT];

enum OutputLightLinguistic { OUTPUT_OFF, OUTPUT_COOL_BOOST,
OUTPUT_WARM_HIGH, OUTPUT_COUNT_L };
float outputMembershipLight[OUTPUT_COUNT_L];

const float outputCentroidsLight[OUTPUT_COUNT_L] = { 0.0, 100.0,
200.0 };

void fuzzifyLight(float lux) {
  if (lux <= 0.0) lightMembership[LIGHT_LOW] = 1.0;
  else lightMembership[LIGHT_LOW] = mu_triangular(lux, LOW_TO_PEAK,
LOW_TO_PEAK, LOW_TO_ZERO);
  lightMembership[LIGHT_MEDIUM] = mu_triangular(lux, MEDIUM_START,
MEDIUM_PEAK, MEDIUM_END);

  if (lux >= HIGH_TO_MAX) lightMembership[LIGHT_HIGH] = 1.0;
  else if (lux < HIGH_START) lightMembership[LIGHT_HIGH] = 0.0;
  else lightMembership[LIGHT_HIGH] = mu_triangular(lux, HIGH_START,
HIGH_TO_MAX, HIGH_TO_MAX);
}

void inferenceLight() {
  for(int i = 0; i < OUTPUT_COUNT_L; i++) outputMembershipLight[i] =
0.0;
  outputMembershipLight[OUTPUT_WARM_HIGH] =
lightMembership[LIGHT_LOW];
```

```
  outputMembershipLight[OUTPUT_COOL_BOOST] =
lightMembership[LIGHT_MEDIUM];
  outputMembershipLight[OUTPUT_OFF] = lightMembership[LIGHT_HIGH];
}

int defuzzifyLight() {
  float numerator = 0.0;
  float denominator = 0.0;
  for (int i = 0; i < OUTPUT_COUNT_L; i++) {
    numerator += outputMembershipLight[i] * outputCentroidsLight[i];
    denominator += outputMembershipLight[i];
  }
  if (denominator == 0.0) return 0;
  int finalPwmOutput = round(numerator / denominator);
  if (finalPwmOutput < 0) finalPwmOutput = 0;
  if (finalPwmOutput > 255) finalPwmOutput = 255;
  return finalPwmOutput;
}

// Function to control RGB LEDs based on a master PWM command (0-
255)
void setRgbPwmFLC(int masterPwmCommand) {
  // Use a simple color mapping based on the FLC output range
  const int R_WARM_LOW = 150; const int G_WARM_LOW = 118; const int
B_WARM_LOW = 88;
  const int R_COOL_MED = 255; const int G_COOL_MED = 200; const int
B_COOL_MED = 150;
  int finalRed = 0; int finalGreen = 0; int finalBlue = 0;

  if (masterPwmCommand < 50) {
    finalRed = 0; finalGreen = 0; finalBlue = 0; // Off
  } else if (masterPwmCommand >= 50 && masterPwmCommand < 150) {
    finalRed = R_COOL_MED; finalGreen = G_COOL_MED; finalBlue =
B_COOL_MED; // Cool Light
  } else if (masterPwmCommand >= 150) {
    finalRed = R_WARM_LOW; finalGreen = G_WARM_LOW; finalBlue =
B_WARM_LOW; // Warm Light
  }

  analogWrite(LED_RED_PIN, finalRed);
  analogWrite(LED_GREEN_PIN, finalGreen);
  analogWrite(LED_BLUE_PIN, finalBlue);
}

// Function used by Blynk manual override
```

```
void setRgbPwmManual(int masterPwm) {
  // Using the same logic as the FLC to ensure consistency
  setRgbPwmFLC(masterPwm);
}

// ====================================================
// --- FLC 2: NOISE CONTROL (MAX9814) CONFIGURATION ---
// ====================================================
const float NOISE_THRESHOLD = 50.0;
const float QUIET_END_MAX = 50.0;
const float QUIET_END_ZERO = 150.0;
const float NORMAL_PEAK = 300.0;
const float NORMAL_END_ZERO_LOW = 100.0;
const float NORMAL_END_ZERO_HIGH = 500.0;
const float LOUD_START_ZERO = 300.0;
const float LOUD_START_MAX = 520.0;

const float VOLUME_LOW_CENTROID = 40.0;
const float VOLUME_MODERATE_CENTROID = 150.0;
const float VOLUME_HIGH_CENTROID = 255.0;
float mu_low_out_N, mu_moderate_out_N, mu_high_out_N;

int read_noise_peak() {
  const int sampleWindow = 50;
  unsigned long startMillis = millis();
  int peakToPeak = 0;
  int signalMax = 0;
  int signalMin = 1023;

  while (millis() - startMillis < sampleWindow) {
    int sample = analogRead(PIN_MAX9814_ANALOG);
    if (sample > signalMax) signalMax = sample;
    else if (sample < signalMin) signalMin = sample;
  }

  peakToPeak = signalMax - signalMin;
  if (peakToPeak < NOISE_THRESHOLD) return 0;
  return peakToPeak;
}

float is_quiet(float peak) {
  if (peak <= QUIET_END_MAX) return 1.0;
  if (peak >= QUIET_END_ZERO) return 0.0;
  return (QUIET_END_ZERO - peak) / (QUIET_END_ZERO - QUIET_END_MAX);
}
```

```
float is_normal_noise(float peak) {
  return mu_triangular(peak, NORMAL_END_ZERO_LOW, NORMAL_PEAK,
NORMAL_END_ZERO_HIGH);
}

float is_loud(float peak) {
  if (peak <= LOUD_START_ZERO) return 0.0;
  if (peak >= LOUD_START_MAX) return 1.0;
  return (peak - LOUD_START_ZERO) / (LOUD_START_MAX -
LOUD_START_ZERO);
}

void inferenceNoise(float mu_quiet, float mu_normal, float mu_loud)
{
  mu_low_out_N = mu_quiet;
  mu_moderate_out_N = mu_normal;
  mu_high_out_N = mu_loud;
}

int defuzzifyNoise(float mu_low_out, float mu_moderate_out, float
mu_high_out) {
  float numerator = mu_low_out * VOLUME_LOW_CENTROID +
mu_moderate_out * VOLUME_MODERATE_CENTROID + mu_high_out *
VOLUME_HIGH_CENTROID;
  float denominator = mu_low_out + mu_moderate_out + mu_high_out;
  if (denominator == 0.0) return 0;
  return round(numerator / denominator);
}


// ======================================================
// --- FLC 3: CLIMATE CONTROL (BME280) CONFIGURATION ---
// ======================================================
const int DEFUZZ_STEPS = 50;
const float PWM_MAX = 255.0;

const float COLD_END_MAX = 0.0;
const float COLD_END_ZERO = 17.0;
const float COMFORT_START_ZERO = 19.0;
const float COMFORT_PEAK = 22.5;
const float COMFORT_END_ZERO = 26.0;
const float HOT_START_ZERO = 25.0;
const float HOT_START_MAX = 30.0;
```

```
const float DRY_END_MAX = 0.0;
const float DRY_END_ZERO = 40.0;
const float HUM_NORMAL_START_ZERO = 30.0;
const float HUM_NORMAL_PEAK = 50.0;
const float HUM_NORMAL_END_ZERO = 70.0;
const float WET_START_ZERO = 60.0;
const float WET_START_MAX = 80.0;

const float FAN_LOW_ZERO_A = 0.0;
const float FAN_LOW_PEAK = 50.0;
const float FAN_LOW_ZERO_B = 100.0;
const float FAN_MEDIUM_ZERO_A = 50.0;
const float FAN_MEDIUM_PEAK = 125.0;
const float FAN_MEDIUM_ZERO_B = 200.0;
const float FAN_HIGH_ZERO = 180.0;
const float FAN_HIGH_MAX = 255.0;


float read_bme280_temperature() {
  float temp = bme.readTemperature();
  if (isnan(temp)) return -999.0;
  return temp;
}

float read_bme280_humidity() {
  float hum = bme.readHumidity();
  if (isnan(hum)) return -999.0;
  return hum;
}

float is_cold(float temp) {
  if (temp <= COLD_END_MAX) return 1.0;
  if (temp >= COLD_END_ZERO) return 0.0;
  return (COLD_END_ZERO - temp) / (COLD_END_ZERO - COLD_END_MAX);
}

float is_comfort(float temp) {
  if (temp <= COMFORT_START_ZERO || temp >= COMFORT_END_ZERO) return
0.0;
  if (temp <= COMFORT_PEAK) return (temp - COMFORT_START_ZERO) /
(COMFORT_PEAK - COMFORT_START_ZERO);
  else return (COMFORT_END_ZERO - temp) / (COMFORT_END_ZERO -
COMFORT_PEAK);
}
```

```
float is_hot(float temp) {
  if (temp <= HOT_START_ZERO) return 0.0;
  if (temp >= HOT_START_MAX) return 1.0;
  return (temp - HOT_START_ZERO) / (HOT_START_MAX - HOT_START_ZERO);
}

float is_dry(float hum) {
  if (hum <= DRY_END_MAX) return 1.0;
  if (hum >= DRY_END_ZERO) return 0.0;
  return (DRY_END_ZERO - hum) / (DRY_END_ZERO - DRY_END_MAX);
}

float is_normal(float hum) {
  if (hum <= HUM_NORMAL_START_ZERO || hum >= HUM_NORMAL_END_ZERO)
return 0.0;
  if (hum <= HUM_NORMAL_PEAK) return (hum - HUM_NORMAL_START_ZERO) /
(HUM_NORMAL_PEAK - HUM_NORMAL_START_ZERO);
  else return (HUM_NORMAL_END_ZERO - hum) / (HUM_NORMAL_END_ZERO -
HUM_NORMAL_PEAK);
}

float is_wet(float hum) {
  if (hum <= WET_START_ZERO) return 0.0;
  if (hum >= WET_START_MAX) return 1.0;
  return (hum - WET_START_ZERO) / (WET_START_MAX - WET_START_ZERO);
}

float is_fan_low(float pwm_value) {
  return mu_triangular(pwm_value, FAN_LOW_ZERO_A, FAN_LOW_PEAK,
FAN_LOW_ZERO_B);
}

float is_fan_medium(float pwm_value) {
  return mu_triangular(pwm_value, FAN_MEDIUM_ZERO_A,
FAN_MEDIUM_PEAK, FAN_MEDIUM_ZERO_B);
}

float is_fan_high(float pwm_value) {
  if (pwm_value <= FAN_HIGH_ZERO) return 0.0;
  if (pwm_value >= FAN_HIGH_MAX) return 1.0;
  return (pwm_value - FAN_HIGH_ZERO) / (FAN_HIGH_MAX -
FAN_HIGH_ZERO);
}
```

```
float fuzzy_inference_and_aggregation_climate(float mu_c, float
mu_n_temp, float mu_h, float mu_d, float mu_n_hum, float mu_w, float
pwm_value) {

  float alpha_R1 = min(mu_c, mu_d);
  float alpha_R4 = min(mu_n_temp, mu_d);
  float alpha_low = max(alpha_R1, alpha_R4);

  float alpha_R2 = min(mu_c, mu_n_hum);
  float alpha_R3 = min(mu_c, mu_w);
  float alpha_R5 = min(mu_n_temp, mu_n_hum);
  float alpha_R7 = min(mu_h, mu_d);
  float alpha_medium = max(max(max(alpha_R2, alpha_R3), alpha_R5),
alpha_R7);

  float alpha_R6 = min(mu_n_temp, mu_w);
  float alpha_R8 = min(mu_h, mu_n_hum);
  float alpha_R9 = min(mu_h, mu_w);
  float alpha_high = max(max(alpha_R6, alpha_R8), alpha_R9);

  float mu_out_low = min(alpha_low, is_fan_low(pwm_value));
  float mu_out_medium = min(alpha_medium, is_fan_medium(pwm_value));
  float mu_out_high = min(alpha_high, is_fan_high(pwm_value));

  return max(max(mu_out_low, mu_out_medium), mu_out_high);
}

int defuzzify_cog_climate(float mu_c, float mu_n_temp, float mu_h,
float mu_d, float mu_n_hum, float mu_w) {
  float numerator = 0.0;
  float denominator = 0.0;
  float step_size = PWM_MAX / (float)DEFUZZ_STEPS;

  for (int i = 0; i <= DEFUZZ_STEPS; i++) {
    float pwm_point = i * step_size;
    float mu_aggregated =
fuzzy_inference_and_aggregation_climate(mu_c, mu_n_temp, mu_h, mu_d,
mu_n_hum, mu_w, pwm_point);
    numerator += mu_aggregated * pwm_point;
    denominator += mu_aggregated;
  }

  if (denominator == 0.0) return 0;
  return (int)(numerator / denominator);
}
```

```
// ========================================================
// --- FLC 4: AIR QUALITY CONTROL (MQ-135) CONFIGURATION ---
// ========================================================
// Crisp Purifier ON/OFF Threshold
const int PURIFIER_THRESHOLD = 350;

// --- 0. CONFIGURABLE MEMBERSHIP FUNCTION PARAMETERS (Analog
Reading) ---
// 1. GAS_LOW
const float LOW_PEAK_AQ = 200.0;
const float LOW_FADE_END_AQ = 250.0;

// 2. GAS_MEDIUM
const float MEDIUM_START_AQ = 210.0;
const float MEDIUM_PEAK_AQ = 280.0;
const float MEDIUM_END_AQ = 350.0;

// 3. GAS_HIGH
const float HIGH_FADE_START_AQ = 320.0;
const float HIGH_PEAK_AQ = 380.0;
const float HIGH_END_AQ = 1023.0;

// --- 1. Input Linguistic Variables (Gas Concentration) ---
enum GasLinguistic {
  GAS_LOW, GAS_MEDIUM, GAS_HIGH, GAS_COUNT
};
float gasMembership[GAS_COUNT];

// --- 2. Output Linguistic Variables (Ventilation Effort - PWM) ---
enum OutputAQLinguistic {
  OUTPUT_LOW_VENT, OUTPUT_MED_VENT, OUTPUT_HIGH_VENT,
OUTPUT_COUNT_AQ
};
float outputMembershipAQ[OUTPUT_COUNT_AQ];

// --- 3. Output Centroids for Defuzzification (CRISP PWM VALUES) --
-
const float outputCentroidsAQ[OUTPUT_COUNT_AQ] = {
  0.0, 127.0, 255.0
};

void fuzzifyGas(float gasValue) {
  // 1. GAS_LOW (Clean Air - Z-Shape Trapezoid)
```

```
  if (gasValue <= LOW_PEAK_AQ) {
    gasMembership[GAS_LOW] = 1.0;
  } else {
    gasMembership[GAS_LOW] = mu_trapezoidal(gasValue, LOW_PEAK_AQ,
LOW_PEAK_AQ, LOW_PEAK_AQ, LOW_FADE_END_AQ);
    if (gasMembership[GAS_LOW] < 0.0) gasMembership[GAS_LOW] = 0.0;
  }

  // 2. GAS_MEDIUM (Moderately Polluted - Triangular Function)
  gasMembership[GAS_MEDIUM] = mu_triangular(gasValue,
MEDIUM_START_AQ, MEDIUM_PEAK_AQ, MEDIUM_END_AQ);

  // 3. GAS_HIGH (Highly Polluted - S-Shape Trapezoid)
  if (gasValue >= HIGH_PEAK_AQ) {
    gasMembership[GAS_HIGH] = 1.0;
  } else {
    gasMembership[GAS_HIGH] = mu_trapezoidal(gasValue,
HIGH_FADE_START_AQ, HIGH_PEAK_AQ, HIGH_PEAK_AQ, HIGH_PEAK_AQ);
    if (gasMembership[GAS_HIGH] < 0.0) gasMembership[GAS_HIGH] =
0.0;
  }
}

void inferenceGas() {
  for(int i = 0; i < OUTPUT_COUNT_AQ; i++) {
    outputMembershipAQ[i] = 0.0;
  }

  // R1: IF Gas is LOW, THEN Ventilation is LOW_VENT (0 PWM)
  outputMembershipAQ[OUTPUT_LOW_VENT] =
max(outputMembershipAQ[OUTPUT_LOW_VENT], gasMembership[GAS_LOW]);

  // R2: IF Gas is MEDIUM, THEN Ventilation is MED_VENT (127 PWM)
  outputMembershipAQ[OUTPUT_MED_VENT] =
max(outputMembershipAQ[OUTPUT_MED_VENT], gasMembership[GAS_MEDIUM]);

  // R3: IF Gas is HIGH, THEN Ventilation is HIGH_VENT (255 PWM)
  outputMembershipAQ[OUTPUT_HIGH_VENT] =
max(outputMembershipAQ[OUTPUT_HIGH_VENT], gasMembership[GAS_HIGH]);
}

int defuzzifyGas() {
  float numerator = 0.0;
  float denominator = 0.0;
```

```
  for (int i = 0; i < OUTPUT_COUNT_AQ; i++) {
    numerator += outputMembershipAQ[i] * outputCentroidsAQ[i];
    denominator += outputMembershipAQ[i];
  }

  if (denominator == 0.0) {
    return 0; // Default to OFF if no rules were fired
  }

  int finalPwmOutput = round(numerator / denominator);

  if (finalPwmOutput < 0) finalPwmOutput = 0;
  if (finalPwmOutput > 255) finalPwmOutput = 255;

  return finalPwmOutput;
}


//
=====================================================================
// --- BLYNK COMMAND PROCESSOR (SERIAL RECEIVER & MANUAL OVERRIDE) -
--
//
=====================================================================
//
/**
 * @brief Processes incoming serial commands from ESP8266/Blynk.
 * Sets the ControlMode flag to MANUAL and stores the new value
immediately.
 */
void processCommand(String command) {
  // Expected command format: V_PIN:VALUE (e.g., V1:180 or V9:1)

  int colonIndex = command.indexOf(':');
  if (colonIndex == -1) return; // Invalid format

  String vPinStr = command.substring(0, colonIndex);
  String valueStr = command.substring(colonIndex + 1);
  int value = valueStr.toInt();

  Serial.print("Rx Cmd: "); Serial.print(vPinStr);
Serial.print("="); Serial.println(value);

  // Set actuator mode to MANUAL, store the new value, and apply
immediately
```

```
  if (vPinStr == "V1") {
    lightMode = MODE_MANUAL;
    manualLightPwm = value;
    setRgbPwmManual(manualLightPwm); // Apply manual change
  } else if (vPinStr == "V3") {
    noiseMode = MODE_MANUAL;
    manualNoisePwm = value;
    analogWrite(PIN_SPEAKER_VOLUME_PWM, manualNoisePwm); // Apply
manual change
  } else if (vPinStr == "V7") {
    fanMode = MODE_MANUAL;
    manualFanPwm = value;
    analogWrite(PIN_FAN_PWM, manualFanPwm); // Apply manual change
  } else if (vPinStr == "V9") {
    purifierMode = MODE_MANUAL;
    manualPurifierAngle = (value == 1) ? 180 : 0;
    PurifierServo.write(manualPurifierAngle); // Apply manual change
  } else if (vPinStr == "V10") {
    ventMode = MODE_MANUAL;
    manualVentAngle = value;
    VentilationServo.write(manualVentAngle); // Apply manual change
  }
}

/**
 * @brief Continuously checks the Hardware Serial buffer (D0/D1) for
commands.
 */
void serialEvent() {
  while (Serial.available()) {
    char inChar = (char)Serial.read();
    inputString += inChar;

    if (inChar == '\n') {
      inputString.trim();
      processCommand(inputString);
      inputString = "";
    }
  }
}

// ================================================
// --- FLC CONTROL LOOP (AUTOMATIC EXECUTION) ---
// ================================================
```

```
long lastControlTime = 0;
const long controlInterval = 1000; // Run FLC every 1 second
(matching the FLC's original delay)

void controlLoopFLC() {
  // Temporarily disable interrupts while reading the volatile
variable
  noInterrupts();
  bool current_occupancy = is_occupied;
  interrupts();

  unsigned long currentMillis = millis();

  // --- FLC 1: LIGHT CONTROL EXECUTION ---
  int light_pwm_flc1 = 0;
  float lux = lightMeter.getLux();
  if (lux >= 0.0) {
    fuzzifyLight(lux);
    inferenceLight();
    light_pwm_flc1 = defuzzifyLight();
  }

  // 1. LIGHT CONTROL ACTUATION (V1 - D9, D10, D11)
  if (lightMode == MODE_FLC) {
    // FLC 5: Occupancy Override
    if (current_occupancy) {
      setRgbPwmFLC(light_pwm_flc1);
    } else {
      setRgbPwmFLC(0); // Override to OFF
    }
  }

  // --- FLC 2: NOISE CONTROL EXECUTION ---
  int noise_peak = read_noise_peak();
  float mu_quiet = is_quiet(noise_peak);
  float mu_normal = is_normal_noise(noise_peak);
  float mu_loud = is_loud(noise_peak);
  inferenceNoise(mu_quiet, mu_normal, mu_loud);
  int volume_pwm_flc2 = defuzzifyNoise(mu_low_out_N,
mu_moderate_out_N, mu_high_out_N);

  // 2. NOISE CONTROL ACTUATION (V3 - D3)
  if (noiseMode == MODE_FLC) {
    // FLC 5: Occupancy Override
    if (current_occupancy) {
```

```
      analogWrite(PIN_SPEAKER_VOLUME_PWM, volume_pwm_flc2);
    } else {
      analogWrite(PIN_SPEAKER_VOLUME_PWM, 0); // Override to OFF
    }
  }

  // --- FLC 3: CLIMATE CONTROL EXECUTION ---
  float current_temp = read_bme280_temperature();
  float current_hum = read_bme280_humidity();
  int pwm_climate = 0;
  float mu_c = 0.0; float mu_n_temp = 0.0; float mu_h = 0.0;
  float mu_d = 0.0; float mu_n_hum = 0.0; float mu_w = 0.0;
  if (current_temp != -999.0 && current_hum != -999.0) {
    mu_c = is_cold(current_temp); mu_n_temp =
is_comfort(current_temp); mu_h = is_hot(current_temp);
    mu_d = is_dry(current_hum); mu_n_hum = is_normal(current_hum);
mu_w = is_wet(current_hum);
    pwm_climate = defuzzify_cog_climate(mu_c, mu_n_temp, mu_h, mu_d,
mu_n_hum, mu_w);
  }

  // --- FLC 4: AIR QUALITY CONTROL EXECUTION (Conditional) ---
  int sensorValue = analogRead(GAS_SENSOR_PIN);
  float gasValue = (float)sensorValue;
  int pwm_air_quality = 0;

  // Purifier Servo (D13) - CRISP LOGIC
  int purifier_angle_flc4 = (sensorValue > PURIFIER_THRESHOLD) ? 180
: 0;

  // Ventilation PWM (FLC 4 Fuzzy Logic) - Conditional Execution
  if (current_occupancy) {
    fuzzifyGas(gasValue);
    inferenceGas();
    pwm_air_quality = defuzzifyGas();
    cached_pwm_air_quality = pwm_air_quality;
    lastAQCheckTime = currentMillis;
  }
  else if (currentMillis - lastAQCheckTime >= AQ_CHECK_INTERVAL_MS)
{
    fuzzifyGas(gasValue);
    inferenceGas();
    pwm_air_quality = defuzzifyGas();
    cached_pwm_air_quality = pwm_air_quality;
    lastAQCheckTime = currentMillis;
```

```
  }
  else {
    pwm_air_quality = cached_pwm_air_quality;
  }

  // FINAL ACTUATION FUSION (FLC 3 + FLC 4)
  int fan_base_pwm = max(pwm_climate, pwm_air_quality);
  int final_pwm_command = 0;

  // FLC 5: Apply Occupancy Override to the Fan/Ventilation System
  if (current_occupancy) {
    final_pwm_command = fan_base_pwm;
  } else {
    final_pwm_command = 0; // Override to OFF
  }

  // 3. FAN CONTROL ACTUATION (V7 - D5)
  if (fanMode == MODE_FLC) {
    analogWrite(PIN_FAN_PWM, final_pwm_command);
  }

  // 4. PURIFIER SERVO ACTUATION (V9 - D13)
  if (purifierMode == MODE_FLC) {
    PurifierServo.write(purifier_angle_flc4);
  }

  // 5. VENTILATION SERVO ACTUATION (V10 - D6)
  if (ventMode == MODE_FLC) {
    int servo_angle = map(final_pwm_command, 0, 255, 0, 180);
    VentilationServo.write(servo_angle);
  }

  // --- DEBUG PRINTING ---
  Serial.println("--------------------------- FLC AUTOMATIC MODE
REPORT --------------------------------");
  Serial.print("| OCC: "); Serial.print(current_occupancy ? "YES" :
"NO");
  Serial.print(" | Light FLC Cmd: "); Serial.print(light_pwm_flc1);
  Serial.print(" | Noise FLC Cmd: "); Serial.print(volume_pwm_flc2);
  Serial.print(" | Climate FLC Cmd (PWM): ");
Serial.print(pwm_climate);
  Serial.print(" | Air FLC Cmd (PWM): ");
Serial.print(pwm_air_quality);
  Serial.print(" | Final Fan/Vent PWM: ");
Serial.println(final_pwm_command);
```

```
  Serial.print("| CONTROL MODES: Light:"); Serial.print(lightMode ==
MODE_FLC ? "FLC" : "MANUAL");
  Serial.print(", Noise:"); Serial.print(noiseMode == MODE_FLC ?
"FLC" : "MANUAL");
  Serial.print(", Fan:"); Serial.print(fanMode == MODE_FLC ? "FLC" :
"MANUAL");
  Serial.print(", Purifier:"); Serial.print(purifierMode == MODE_FLC
? "FLC" : "MANUAL");
  Serial.print(", Vent:"); Serial.println(ventMode == MODE_FLC ?
"FLC" : "MANUAL");
  Serial.println("-----------------------------------------------------
-----------------------------------");
}

// ==============================
// --- ARDUINO SETUP AND LOOP ---
// ==============================

void setup() {
  // Starting Hardware Serial communication with ESP8266 and debug
  Serial.begin(BAUD_RATE);
  Serial.println("--- ATmega Combined FLC + Blynk Controller
Initializing ---");

  // 1. Initializing BH1750 (Light FLC)
  if (!lightMeter.init()) { Serial.println(" BH1750 FAILED."); }
  else { lightMeter.setMode(0x10); }

  // 2. Initializing BME280 (Climate FLC)
  if (!bme.begin(BME_ADDRESS)) {
    Serial.println(" BME280 FAILED. Check I2C wiring (A4/A5) or
address.");
  }

  // 3. Setup Actuator Pins
  pinMode(LED_RED_PIN, OUTPUT);
  pinMode(LED_GREEN_PIN, OUTPUT);
  pinMode(LED_BLUE_PIN, OUTPUT);
  pinMode(PIN_SPEAKER_VOLUME_PWM, OUTPUT);
  pinMode(PIN_FAN_PWM, OUTPUT); // D5

  // FLC 5: PIR Sensor Setup & Interrupt
  pinMode(PIN_PIR_SENSOR, INPUT_PULLUP);
  is_occupied = (digitalRead(PIN_PIR_SENSOR) == HIGH);
```

```
  attachInterrupt(digitalPinToInterrupt(PIN_PIR_SENSOR),
occupancy_change_isr, CHANGE);

  // Servo Setup: Attaching Servos and setting initial positions
  PurifierServo.attach(PURIFIER_SERVO_PIN); // D13
  VentilationServo.attach(PIN_VENTILATION_SERVO); // D6

  // Initial State: All actuators OFF/LOW (0)
  setRgbPwmManual(0);
  analogWrite(PIN_SPEAKER_VOLUME_PWM, 0);
  PurifierServo.write(0);
  VentilationServo.write(0);
  analogWrite(PIN_FAN_PWM, 0);

  Serial.println("System Ready.");
}

void loop() {
  serialEvent(); // MUST run continuously to catch immediate Blynk
commands (Manual Override)

  // Running the FLC control loop periodically
  if (millis() - lastControlTime >= controlInterval) {
    controlLoopFLC();
    lastControlTime = millis();
  }
}
```

ESP8266 Code:

```
// ========================================
// --- BLYNK CREDENTIALS & LIBRARY SETUP ---
// ========================================
#define BLYNK_TEMPLATE_ID   "TMPL6haRBSfNB"
#define BLYNK_TEMPLATE_NAME "Animo Platte"
#define BLYNK_AUTH_TOKEN    "VR6zxAtYPdT2GverBZ17kgGWV8nm4i6x"

#define BLYNK_PRINT Serial  // Debug output via USB
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// Wi-Fi credentials
char ssid[] = "DMTs-WiFi";
char pass[] = "";
```

```
// Communication Baud Rate
#define BAUD_RATE 115200

// ============================================
// --- BLYNK WRITE HANDLERS (PROTOCOL SENDER) ---
// ============================================
// Protocol: Sends command as "V_PIN:VALUE\n" to the ATmega.

// 1. Light Master PWM (V1) -> ATmega Pin D9 (Master Brightness)
BLYNK_WRITE(V1) {
  int value = param.asInt();
  // Command format: "V1:VALUE\n"
  String command = "V1:" + String(value) + "\n";
  Serial.print(command);
  Blynk.logEvent("light_update", "Light PWM set to " +
String(value));
}

// 3. Noise PWM (V3) -> ATmega Pin D3
BLYNK_WRITE(V3) {
  int value = param.asInt();
  // Command format: "V3:VALUE\n"
  String command = "V3:" + String(value) + "\n";
  Serial.print(command);
  Blynk.logEvent("noise_update", "Noise PWM set to " +
String(value));
}

// 4. Fan PWM (V7) -> ATmega Pin D5
BLYNK_WRITE(V7) {
  int value = param.asInt();
  // Command format: "V7:VALUE\n"
  String command = "V7:" + String(value) + "\n";
  Serial.print(command);
  Blynk.logEvent("fan_update", "Fan PWM set to " + String(value));
}

// 2. Air Purifier Servo ON/OFF (V9) -> ATmega Pin D13 (Digital)
BLYNK_WRITE(V9) {
  int value = param.asInt(); // 1 or 0
  // Command format: "V9:VALUE\n"
  String command = "V9:" + String(value) + "\n";
  Serial.print(command);
```

```
  // FIX: Use String() constructor to enable concatenation of C-strings
  Blynk.logEvent("air_purifier_update", String("Air Purifier state set to ") + (value ? "ON" : "OFF"));
}

// 5. Ventilation Servo Position (V10) -> ATmega Pin D6
BLYNK_WRITE(V10) {
  int value = param.asInt(); // Servo angle (0-180)
  // Command format: "V10:VALUE\n"
  String command = "V10:" + String(value) + "\n";
  Serial.print(command);
  Blynk.logEvent("vent_update", "Ventilation Servo angle set to " + String(value));
}

// =======================
// --- SETUP AND LOOP ---
// =======================

void setup() {
  // Use Hardware Serial (USB) for debug output
  Serial.begin(BAUD_RATE);
  delay(10);

  Serial.println("\n--- ESP8266 Blynk Handler Initializing ---");

  // Begin Blynk connection
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

void loop() {
  Blynk.run();
}
```

C. System Images

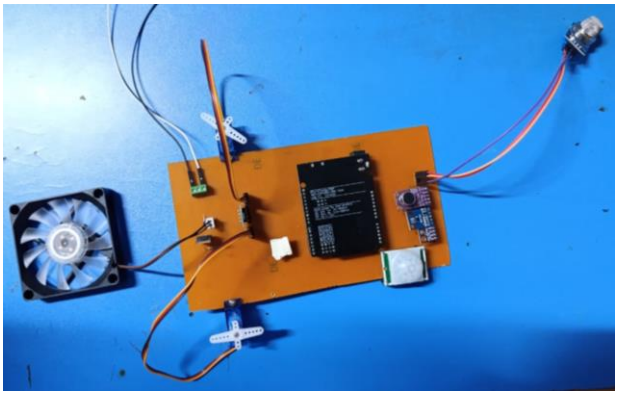


Figure 12. The Final Assembly a)

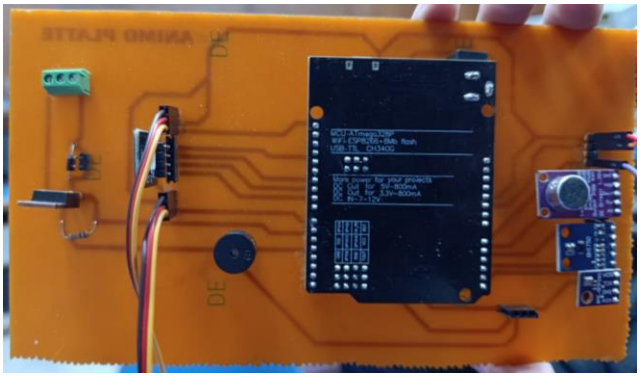


Figure 12. b)

Figure 12. c)

D. Blynk Console

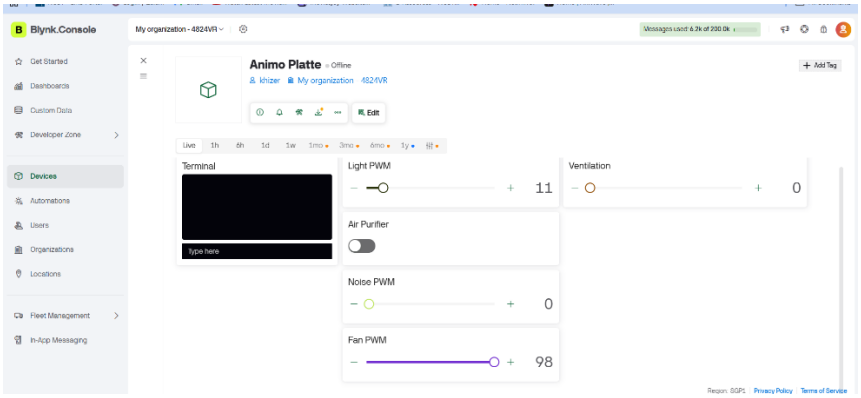


Figure 13. Blynk Console

E. Demo Video

Links:

https://youtube.com/shorts/jURIiFCFpSs?si=rFn-kbtrB40PKjp7

https://youtube.com/shorts/masb5z87Das?si=v_qwKbgJOXyG0rk0